

WhatsApp Transport Technical Plan

Adding WhatsApp as a second channel to the existing engine: Meta platform requirements, the transport design, the 24-hour window, hosting options, and a full cost breakdown.

Prepared	16 September 2026
Basis	The assistant as deployed (commit <code>35b97e4</code>); Meta WhatsApp Business Platform, Cloud API
Companion	<i>MKY-Booking-Assistant-Technical.pdf</i> describes the existing system this plan extends

Contents

1. Scope and approach
2. Meta platform requirements and timelines
3. WhatsApp Cloud API behaviour that shapes the transport
4. Transport design: `api/whatsapp.js` and `lib/whatsapp.js`
5. Mapping the engine's messages to WhatsApp
6. The 24-hour window and templates in the outbox
7. Identity, sessions and the data model
8. Hosting: Vercel and the alternatives
9. Cost breakdown
10. Privacy and data handling
11. Testing
12. Work plan and effort

1. Scope and approach

The engine (`lib/flow/*`), the booking rules, the readers, the document pipeline, the outbox, the console and the 237 tests are channel-independent: they consume inputs of the kinds `command`, `callback`, `text`, `contact`, `caption` and `document`, and produce messages carrying text plus inline keyboard rows. Telegram is one transport (`api/telegram.js`) that turns Telegram updates into those inputs and those messages into Telegram calls. WhatsApp is a second transport doing the same for Meta's Cloud API. Both run at once; every booking, document and ticket already records its `channel`.

Nothing in the engine changes. The work is one webhook, one API client, one keyboard adapter, template sending in the outbox drain, one small migration, and the transport's tests.

2. Meta platform requirements and timelines

REQUIREMENT	DETAIL	TIME
Meta Business Portfolio	MKY's business account; owner is a company representative with a Facebook login. Everything else hangs off it.	Same day
WhatsApp Business Account (WABA)	Created via the WhatsApp Business Platform onboarding in the portfolio. Holds the phone number(s), templates, quality rating and messaging limits.	Same day
Phone number	Not registered on WhatsApp / WhatsApp Business app. Verified by SMS or voice call. A number in the phone app can be migrated (it leaves the app). Landlines are fine.	Same day

Meta app, WhatsApp product	Developer app in the same portfolio. Yields the phone number id, WABA id, app secret, and a permanent access token issued to a <i>system user</i> with <code>whatsapp_business_messaging</code> and <code>whatsapp_business_management</code> . App Review is not required when the app messages only the portfolio's own WABA.	Half a day
Display name review	Must reflect the business; reviewed by Meta.	1–2 business days
Business verification	Legal name, registration/tax document, matching address, company-domain email or website. Unverified: 250 unique business-initiated recipients per rolling 24 h and no template use beyond that. Verified: tier starts at 1,000/day and scales to 10K, 100K, unlimited with quality.	2–10 business days; up to ~3 weeks
Templates	Utility category, per language. Reviewed by Meta; edits require re-review.	Minutes to 48 h
Privacy policy URL	Required on the app. Content in section 10.	A day
Test number	Meta provides a free test phone number that can message up to five verified recipients before verification. Development and end-to-end tests run against it.	Immediately
Optional: Official Business Account	The green tick. Applied for after verification; not needed for messaging.	Days to weeks

Direct vs Business Solution Provider

The Cloud API is used directly (no fee beyond per-message prices) or through a BSP — Twilio, 360dialog, Infobip — that fronts the same API with its own SDK and billing (about \$0.005 per message, or ~€49–100 per number per month). The transport below targets the Cloud API; a BSP changes only the base URL, authentication and the inbound payload envelope. Recommendation: direct.

3. Cloud API behaviour that shapes the transport

BEHAVIOUR	CONSEQUENCE FOR THE DESIGN
Webhook delivery and retries	Meta POSTs to the webhook and retries (with backoff, for hours) when it does not get a 2xx quickly. Same shape as Telegram: claim the message id, answer 200 at once, do the slow part in the background. <code>processed_updates</code> and <code>waitUntil</code> apply unchanged.
Signature	Every POST carries <code>X-Hub-Signature-256</code> = HMAC-SHA256(raw body, app secret). Verified on the raw body before parsing. The GET verification handshake echoes <code>hub.challenge</code> when <code>hub.verify_token</code> matches.
Message kinds inbound	<code>text</code> , <code>interactive</code> (<code>button_reply</code> / <code>list_reply</code> with the id we set), <code>document</code> , <code>image</code> , <code>contacts</code> , <code>location</code> , <code>reaction</code> , plus <code>statuses</code> (sent/delivered/read) for our outbound messages. Statuses and reactions are acknowledged and ignored; delivered/read could later feed the outbox's delivery state.
Interactive limits	Reply buttons: max 3 per message, title ≤ 20 characters, id ≤ 256 bytes. List: one button opening ≤ 10 rows in sections, row title ≤ 24 characters. No inline URL buttons except CTA-URL messages. Sent messages cannot be edited or have buttons removed.
Media	Inbound media arrives as an id; fetch its URL with the token, then download (URL is short-lived; media retained ~ 30 days). Documents ≤ 100 MB, images ≤ 5 MB. Outbound: upload to get a media id, or send by URL. Captions on inbound media are delivered with the message.
No albums	Several files sent at once arrive as separate messages within a second, with no group id. The batch logic (record first, read in parallel, last to finish speaks, claimed reply) works as is; the album grace wait becomes a short fixed wait on every document.
24-hour customer service window	Free-form messages only within 24 h of the client's last message; outside it, only approved templates (section 6). Template messages are billed per message by category and recipient country; utility templates inside the window are free.
Identity	The sender is identified by <code>wa_id</code> (the phone number in E.164 without the plus) and a profile name. No username, no chat id distinct from the number.
Typing indicator and read receipts	Supported via the messages endpoint (mark as read, typing indicator) — used in place of Telegram's <code>sendChatAction</code> .
No pinning, no commands	The pinned status card has no equivalent; slash commands are plain text.

4. Transport design

4.1 api/whatsapp.js

```
GET /api/whatsapp?hub.mode=subscribe&hub.verify_token=...&hub.challenge=... → 200 hub.challenge
POST /api/whatsapp
  1. read the raw body; verify X-Hub-Signature-256 with WHATSAPP_APP_SECRET; else 401
  2. for each entry.changes[].value:
      for each messages[]:      claimUpdate(message.id, wa_id) → readInput → dispatch
                                statuses[] / errors[]:      log, and update outbox delivery state (later)
  3. answer 200 immediately; documents continue under waitUntil (as api/telegram.js does)
```

```
{ "entry": [{ "changes": [{ "value": {
  "metadata": { "phone_number_id": "..."},
  "contacts": [{ "wa_id": "201005551234", "profile": { "name": "Ariful" } }],
  "messages": [
    { "id": "wamid....", "from": "201005551234", "type": "text",
      "text": { "body": "Nile Motors and my number is +20 100 555 1234" } },
    { "id": "wamid....", "from": "201005551234", "type": "interactive",
      "interactive": { "type": "button_reply", "button_reply": { "id": "bk:confirm", "title": "Confirm" } } },
    { "id": "wamid....", "from": "201005551234", "type": "document",
      "document": { "id": "MEDIA_ID", "filename": "kitJ-invoice.pdf", "mime_type": "application/pdf",
                    "caption": "Rotterdam to Damietta. DAF XF 480 FT, chassis XLRTEH4350G741552" } }
  ] } } ] } }
```

4.2 readInput mapping

INBOUND	ENGINE INPUT
text "menu", "start", "cancel", "reset" (and Arabic forms)	{ kind: 'command', command: '/menu' '/start' '/cancel' '/reset' }
other text	{ kind: 'text', text }
interactive.button_repl y.id / list_reply.id	{ kind: 'callback', callback: parseCallback(id) } — the ids are the existing payloads (menu:book , bk:mrn:existing , ct:urgent:yes ...)
contacts[0].phones[0].w a_id	{ kind: 'contact', phone }
document / image	validate → beginDocument → 200 → background: { kind: 'caption' } if a caption, download by media id, completeDocument, batch decision, { kind: 'document', speak, batch }
statuses, reaction, location, unsupported	ignored (200)

4.3 lib/whatsapp.js

```
call(path, payload)          POST https://graph.facebook.com/v21.0/{PHONE_NUMBER_ID}/{path}, Bearer token
sendText(to, body)           messages: { type: 'text', text: { body, preview_url: false } }
sendButtons(to, body, buttons) interactive.type = 'button', ≤ 3 × { id, title(≤20) }
sendList(to, body, rows)      interactive.type = 'list', button 'Choose', ≤ 10 rows { id, title(≤24) }
sendDocument(to, buffer, name, caption) POST /media (multipart) → id; messages { type: 'document', document: {
sendContact(to, name, phone)   messages { type: 'contacts' } – the direct line as a tappable card
sendTemplate(to, name, lang, components) messages { type: 'template' }
markRead(messageId) · typing(messageId) messages { status: 'read', typing_indicator: { type: 'text' } }
mediaUrl(id) → download(url)   GET /{id} → { url } ; GET url with the token → bytes
```

Like `lib/telegram.js`: plain `fetch`, no SDK, every call logs a non-2xx and returns the body; nothing here throws into the flow.

5. Mapping the engine's messages to WhatsApp

Handlers return messages as `{ text, inline?: rows, keyboard?, oneTime? }`. An adapter in the transport renders them:

ENGINE MESSAGE	WHATSAPP RENDERING
Text, no buttons	<code>sendText</code> . Bilingual bodies are fine; WhatsApp renders Arabic right-to-left per line.
Inline rows with ≤ 3 buttons	<code>sendButtons</code> . Titles are the label with the emoji, truncated to 20 characters; because the label carries both languages, the adapter uses a short per-language title table keyed by callback payload (e.g. <code>menu:book</code> → "📖 احجز / Book").
Inline rows with 4–10 buttons (edit menu, document classify)	<code>sendList</code> with row titles from the same table; the list button reads "اختار / Choose".
More than 10 (does not occur today)	Split across two list messages.
Reply keyboard "Share my number"	Not sent. <code>askNextBasic</code> for the phone step on WhatsApp offers <code>[Use +20...]</code> <code>[Another number]</code> buttons; "Another number" → the client types it.
Documents (the PDF copy)	<code>sendDocument</code> , caption as today.
Direct line after hours	The text, plus <code>sendContact</code> so it is one tap to call.
Typing	<code>markRead</code> + typing indicator on the inbound message id, in place of <code>sendTyping</code> .
<code>clearButtons</code>	No-op; the engine already refuses stale taps with a clear message.

Truncation is asserted at build time the way `cb()` asserts Telegram's 64-byte payload limit today: a title over 20 characters throws in development rather than being silently rejected in production.

6. The 24-hour window and templates in the outbox

- `conversation_sessions` gains `last_client_message_at`, stamped on every inbound message.
`windowOpen(session) = now - last_client_message_at < whatsapp_window_hours` (setting, default 24).
- The outbox drain, for a row with `channel = 'whatsapp'`: inside the window → render as today and send text/document; outside → look up the template for `event_type` in `bot_settings.whatsapp_templates` and `sendTemplate` with the payload's fields as components. The PDF events use a template with a document header.
- Template set (utility, `ar` and `en`): `booking_confirmed` (document header: the confirmation PDF; body: reference, shipment id), `missing_information_requested` (reference, what), `booking_rejected` (reference, reason), `mrn_issued` (mrn, reference), `shipment_update` (shipment, text), `operations_message` (reference, text). Each with one quick-reply button "Reply / رد" so the client's tap reopens the window.

- Language choice per client: the language of their last message (Arabic script → `ar`), stored on the client record; falls back to `en`.
- Failures Meta reports through `statuses[].errors` (e.g. 131047 re-engagement, 131026 not a WhatsApp user) map to the outbox's permanent-failure list, as Telegram's "bot was blocked" does today.

7. Identity, sessions and the data model

- Session key `whatsapp:<wa_id>`; `ctx = { channel: 'whatsapp', chatId: wa_id, clientId, userName: profile.name }`.
- Migration: `clients.whatsapp_id` text unique; `conversation_sessions.last_client_message_at` timestampz; `bot_settings` rows `whatsapp_templates`, `whatsapp_window_hours`. Bookings, documents, tickets and tasks already carry `channel` and `chat_id`.
- `upsertWhatsAppClient({ waId, profileName })` mirrors `upsertTelegramClient`; `phoneOnFile()` returns the WhatsApp number first, so step 1 offers it and "Talk to an agent" never asks.
- A person who uses both channels is two client rows unless linked by number; `phoneOnFile()` already matches by phone, so a Telegram client who gave the same number is recognised.
- The pinned card (`lib/pinned.js`) stays Telegram-only; on WhatsApp the same events send a status text inside the window or the `shipment_update` template outside it.

8. Hosting: Vercel and the alternatives

The application is plain Node.js with no framework and no Vercel-specific API beyond `waitUntil`, so it can run on any host that serves HTTPS and runs Node 20. The database and storage stay on Supabase in every option; only the code moves.

OPTION	PER MONTH	FIT	WHAT CHANGES TECHNICALLY
Vercel Hobby (today)	\$0	Not for production: the plan's terms are for personal, non-commercial use; 12-function cap (reached); cold starts after idle; daily cron at most.	To add <code>/api/whatsapp</code> at all, fold <code>api/status.js</code> into <code>api/health.js</code> .
Vercel Pro	\$20 / member	Least change. Function cap lifted, Fluid compute, cron every minute for the outbox, longer durations, Frankfurt region as now.	None.
Fly.io	≈ \$3–8	Always-on small VM in Frankfurt (<code>fra</code>), auto-restart, built-in TLS. No cold starts.	A 40-line <code>server.js</code> routing <code>/api/*</code> to the existing handlers (they are <code>(req, res)</code> functions); a Dockerfile; <code>waitUntil</code> becomes a no-op (the process lives on); outbox retries by a <code>setInterval</code> or Fly's scheduled machines.
Railway	≈ \$5–10	Same shape as Fly.io, simpler dashboard; EU region (Amsterdam).	As Fly.io; deploy from the GitHub repository directly.
Render	\$7 (Starter)	Always-on web service; Frankfurt region available.	As Fly.io.
Hetzner VPS (Germany)	≈ €4–8	Cheapest and fully controlled. Requires someone to run it: OS updates, Node updates, process manager, TLS (Caddy), monitoring, backups of nothing (data is on Supabase).	As Fly.io plus Docker Compose or PM2 and Caddy; deploy by <code>git pull</code> or a GitHub Action over SSH.
Cloudflare Workers / Supabase Edge Functions	\$0–5	Not suitable without rework: <code>pdfjs-dist</code> with <code>@napi-rs/canvas</code> , and <code>pdfkit</code> , need Node's native modules, which these runtimes do not provide.	Would require moving document reading and PDF generation elsewhere.

Recommendation. Vercel Pro for the least disruption and the same operational model as today. Fly.io or Railway if a lower fixed cost matters and a few days of migration are acceptable; the code needs only a small server wrapper. A Hetzner VPS only if MKY already has someone who administers servers.

9. Cost breakdown

9.1 One-time

ITEM	COST	NOTES
Meta accounts, verification, number registration	\$0	Meta charges nothing.
Phone number	operator's rate	A SIM or landline; one-time and monthly per the operator.
Engineering: the WhatsApp transport, templates, tests, pilot	≈ 2 weeks of work	Section 12. Priced at the agreed rate.
Migration to another host (only if chosen)	≈ 2–4 days of work	Section 8.

9.2 Fixed, per month

ITEM	PER MONTH	NOTES
Hosting: Vercel Pro (or an always-on service, \$5–10; or Hobby \$0 for testing only)	\$20	Section 8.
Supabase Pro	\$25	8 GB database, 100 GB storage, 250 GB egress, daily backups, no pausing. The free tier pauses after 7 idle days and caps storage at 1 GB; usable for testing only.
WhatsApp Cloud API	\$0	No platform fee.
BSP fee (only via a provider)	\$50–100	Twilio has no monthly fee but charges ≈ \$0.005 per message; 360dialog ≈ €49 per number.
Resend	\$0	3,000 emails / month free; \$20 beyond.
Telegram	\$0	
Fixed total, direct with Meta, Vercel Pro + Supabase Pro	\$45	Plus the phone line.

9.3 Per use

ITEM	PRICE	NOTES
WhatsApp service conversations (client-initiated, and replies within 24 h)	free	All bookings, tracking and questions.

WhatsApp utility template, recipient in Egypt	≈ \$0.01–0.02	Free when sent inside an open window. Indicative; the rate card at business.whatsapp.com/products/platform-pricing governs.
WhatsApp marketing template, Egypt	≈ \$0.10	Not used by the assistant; listed for completeness.
OpenAI gpt-4.1-mini (extraction, reading, NLU)	\$0.40 / M input, \$1.60 / M output tokens	A text-layer PDF page ≈ 1–2K tokens; a scanned page via vision ≈ 1–3K tokens plus the image. Three text PDFs ≈ 1–2 cents; three scans ≈ 5–10 cents.
OpenAI gpt-4.1 (questions)	\$2 / M input, \$8 / M output	≈ 1–2 cents per answered question with the tool sheet.
OpenAI embeddings (knowledge base)	negligible	Ingestion only.
Supabase storage egress beyond the plan	\$0.09 / GB	PDFs are small; not a factor at this scale.

9.4 Scenarios (both channels, direct with Meta)

BOOKINGS / MONTH	FIXED	TEMPLATES (3 / BOOKING)	MODEL USAGE	TOTAL
100	\$45	≈ \$5	≈ \$10	≈ \$60
300	\$45	≈ \$15	≈ \$30	≈ \$90
1,000	\$45	≈ \$45	≈ \$100	≈ \$190

Assumptions: three utility templates per booking, half of them inside the window; one or two free questions per booking; a mix of text and scanned PDFs. Telegram traffic adds only model usage. At 1,000 bookings a month the Vercel Pro included function invocations and Supabase Pro limits are not approached.

10. Privacy and data handling

- **Stored by MKY** (Supabase, AWS eu-central-1): client name and number, vehicle and route, documents and what was read from them, the conversation transcript used for question answering, tickets, the audit trail. Row Level Security on; server-only key.
- **Meta:** messages and media in transit and for a limited retention; message metadata (delivery, read). Meta's WhatsApp Business terms and data processing terms apply; Meta does not receive the derived booking data.
- **OpenAI:** document text/images and message text, under API data-usage terms (no training on API data); outputs validated by code before storage.
- **Privacy policy page** (required by Meta): what is collected; purpose (booking and shipment handling); storage location and retention; sub-processors (Meta, Supabase, Vercel or the chosen host, OpenAI, Resend); how to request deletion. Deletion = client row, bookings, documents (storage objects), tickets, sessions, transcript.
- **Retention:** a setting per table (e.g. transcripts 90 days after the last message; documents 12 months after delivery) applied by a scheduled job — a decision for MKY.
- **Egypt PDPL (Law 151/2020):** EU storage, a policy, a deletion path and purpose limitation cover the practical duties; controller registration should be confirmed by MKY's counsel.

11. Testing

- The engine suite (237 tests) is unchanged.
- `tests/whatsapp.test.mjs` : signature verification (valid, tampered, missing); the GET handshake; recorded payloads for text, button reply, list reply, contact, a document with caption, three documents in a burst (one reply), a status update (ignored); the keyboard adapter's button/list decision and title limits; the outbox's window decision and template rendering, with a fake Graph API recording calls.
- End-to-end against Meta's test number before verification; then the pilot on the real number.

12. Work plan and effort

WEEK	ENGINEERING	MKY
1	Transport: webhook, signature, readInput, API client, keyboard adapter, phone step, sessions, migration. Templates written and submitted.	Accounts, number, verification, display name, payment method, privacy page, hosting decision.
2	Outbox window logic and template sending; status messages in place of the pinned card; transport tests; hosting change if chosen.	Respond to Meta; approve template wording.
3	Cut over the verified number; pilot monitoring; fixes.	Pilot with two or three clients.
4	Hand-over; monitoring of quality rating and costs.	Go live.

Engineering effort: about two working weeks for the transport, templates and tests; two to four days more for a host migration. The calendar is set by Meta's verification, which runs in parallel.